Experiment 10: A Search Algorithm*

Aim

To write a Python program to implement the A* Search Algorithm.

Procedure

1. Create the graph with edge costs.
2. Define heuristic values for each node.
3. Select the start and goal nodes.
4. Calculate the evaluation function:
   f(n) = g(n) + h(n).
5. Expand the node having the smallest f(n).
6. Continue until the goal node is reached.
7. Display the shortest path.

Source Code

main.py

Visualize

Run ▶

```python
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'D': 2, 'E': 5},
    'C': {'F': 3},
    'D': {},
    'E': {'F': 1},
    'F': {}
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 1,
    'E': 1,
    'F': 0
}

open_list = ['A']
closed_list = []

g = {'A': 0}
parent = {'A': 'A'}

goal = 'F'

while open_list:
    n = min(open_list, key=lambda x: g[x] + heuristic[x])

    if n == goal:
        path = []

        while parent[n] != n:
            path.append(n)
```

Output

```
Path: ['A', 'C', 'F']
Cost: 7
```

main.py Visualize Run ▶ Output

```
18
19  open_list = ['A']
20  closed_list = []
21
22  g = {'A': 0}
23  parent = {'A': 'A'}
24
25  goal = 'F'
26
27  while open_list:
28      n = min(open_list, key=lambda x: g[x] + heuristic[x])
29
30      if n == goal:
31          path = []
32
33          while parent[n] != n:
34              path.append(n)
35              n = parent[n]
36
37          path.append('A')
38          path.reverse()
39
40          print("Path:", path)
41          print("Cost:", g[goal])
42          break
43
44      open_list.remove(n)
45      closed_list.append(n)
46
47      for m, weight in graph[n].items():
48          if m not in open_list and m not in closed_list:
49              open_list.append(m)
50              parent[m] = n
51              g[m] = g[n] + weight
```

```
Path: ['A', 'C', 'F']
Cost: 7
```

Result

Thus, the A* Search Algorithm was implemented successfully and the shortest path from the start node to the goal node was obtained.